

Competitive Programming

Section 34 Editorial: Probability and Expected Value

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Expected Prize	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Full C++ solution	4
2.5	Full Python solution	5
3	B. First Success	6
3.1	Restatement	6
3.2	Algorithm and proof	6
3.3	Full C++ solution	6
3.4	Full Python solution	7
4	C. Coupon Collector	8
4.1	Restatement	8
4.2	Algorithm	8
4.3	Correctness proof	8
4.4	Full C++ solution	8
4.5	Full Python solution	9
5	Testing discussion	10
6	What to carry forward	11

1 Editorial overview

Problem	Pattern	Formula
A. Expected Prize	linearity	$\sum v_i p_i/q_i$
B. First Success	geometric wait	q/p
C. Coupon Collector	staged waits	$n * H_n$

2 A. Expected Prize

2.1 Restatement

Each event gives $v[i]$ points with probability $p[i]/q[i]$. Compute expected total points modulo MOD.

2.2 Algorithm

For each event:

```
contribution = v[i] * p[i] * inverse(q[i])
```

Sum all contributions.

2.3 Correctness proof

Let X_i be the points from event i . It equals $v[i]$ with probability $p[i]/q[i]$ and 0 otherwise, so $E[X_i] = v[i]p[i]/q[i]$. By linearity of expectation, the expected total is the sum of these values. Modular inverses represent the rational divisions exactly under prime modulus.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    long long ans = 0;
    for (int i = 0; i < n; i++) {
        long long p, q, v;
        cin >> p >> q >> v;
        ans = (ans + (v % MOD) * (p % MOD) % MOD * mod_pow(q, MOD - 2)) % MOD;
    }
    cout << ans << '\n';
    return 0;
}

```

2.5 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    idx = 1
    ans = 0
    for _ in range(n):
        p, q, v = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        ans = (ans + v % MOD * p % MOD * pow(q, MOD - 2, MOD)) % MOD
    print(ans)

```

3 B. First Success

3.1 Restatement

Repeated independent attempts succeed with probability p/q . Compute the expected number of attempts until the first success.

3.2 Algorithm and proof

Let E be the expected number of attempts. One attempt is always made. If it fails, with probability $1-p/q$, the process restarts.

$$\begin{aligned} E &= 1 + (1-p/q)E \\ (p/q)E &= 1 \\ E &= q/p \end{aligned}$$

So print $q * \text{inverse}(p)$.

3.3 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        long long p, q;
```

```

    cin >> p >> q;
    cout << q % MOD * mod_pow(p, MOD - 2) % MOD << '\n';
}
return 0;
}

```

3.4 Full Python solution

```

import sys

MOD = 1_000_000_007

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    p, q = data[idx], data[idx + 1]
    idx += 2
    out.append(str(q % MOD * pow(p, MOD - 2, MOD) % MOD))
print("\n".join(out))

```

4 C. Coupon Collector

4.1 Restatement

With n equally likely coupon types, compute the expected number of draws to collect all types.

4.2 Algorithm

When i types are already collected, the probability of getting a new type is $(n-i)/n$, so the expected waiting time for the next new type is $n/(n-i)$.

Summing:

```
answer = n * (1 + 1/2 + ... + 1/n)
```

Precompute modular harmonic sums.

4.3 Correctness proof

The collection process splits into n stages: going from i collected types to $i+1$. Each stage is a geometric wait with success probability $(n-i)/n$, so its expected length is $n/(n-i)$. Expectations add, giving the formula. $\square$

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
```



```

ios::sync_with_stdio(false);
cin.tie(nullptr);
int q;
if (!(cin >> q)) return 0;
vector<int> values(q);
int limit = 0;
for (int &n : values) {
    cin >> n;
    limit = max(limit, n);
}
vector<long long> harmonic(limit + 1, 0);
for (int i = 1; i <= limit; i++) {
    harmonic[i] = (harmonic[i - 1] + mod_pow(i, MOD - 2)) % MOD;
}
for (int n : values) cout << n * harmonic[n] % MOD << '\n';
return 0;
}

```

4.5 Full Python solution

```

import sys

MOD = 1_000_000_007
data = list(map(int, sys.stdin.buffer.read().split()))
q = data[0]
values = data[1 : 1 + q]
limit = max(values)
harmonic = [0] * (limit + 1)
for i in range(1, limit + 1):
    harmonic[i] = (harmonic[i - 1] + pow(i, MOD - 2, MOD)) % MOD
print("\n".join(str(n * harmonic[n] % MOD) for n in values))

```

5 Testing discussion

Random tests use exact rational arithmetic with Python `Fraction`, then convert the result to modular form. This catches the common mistake of using floating point or rounding.

6 What to carry forward

Expected value problems are often easier than distribution problems. Try linearity first; if the process has repeated waiting for success, look for a geometric stage.